

leading to

$$\Omega(\theta) = \frac{1}{N} \sum_{i=1}^N \left(b - \frac{1}{K} \sum_{j=1}^K \log(\sigma_{i,j}) \right)^+. \quad (6)$$

With this regularizer, inlier samples pay the entropy cost in order to produce a clean feature for classification to satisfy the supervised learning loss. Outlier samples, caused by either label noise or being out-of-distribution, are anyway hard to classify; the regularizer naturally inflates their entropy since doing so does not impact the supervised learning loss. This high-variance representation in turn reduces their (negative) impact on network fitting.

Adversarial Defense To defend against adversarial samples, we aim to inflate entropy over both the neuron- and sample-axes to produce a highly stochastic model:

$$\Omega(\theta) = \frac{1}{N} \frac{1}{K} \sum_{i=1}^N \sum_{j=1}^K (b - \log(\sigma_{i,j}))^+. \quad (7)$$

This looks similar to VIB’s adversarial defense (Alemi et al. 2017), but with the vital difference that our entropy regularizer is not restricted to the $\mathcal{N}(0, 1)$ prior used in VIB. It can be seen as learning a layer-wise data augmentation policy, which turns out to be very useful for adversarial defense in practice.

Calibration For training an easy-to-calibrate model, we use the instance-wise entropy regularizer in Eq. 6, which we expect to assign more variance to instances that are hard to classify. Recall that SE-SNN produces data-dependent noise variance $\sigma_{i,j}$ for the i -th sample and j -th neuron, which can be considered an indicator of confidence. We now link this to temperature calibration, which is a simple and widely used method to calibrating modern neural networks (Guo et al. 2017). Temperature-scaling based calibration uses a scalar parameter T for all classes to raise the output entropy. We further exploit our variance estimate to produce better temperature calibrated confidences p_i from logits l_i as

$$p_i = \text{softmax}(l_i / (T \sum_{j=1}^K \sigma_{i,j})). \quad (8)$$

As per standard temperature calibration (Guo et al. 2017), T is optimized with respect to validation negative log likelihood. Neither standard temperature calibration, nor our generalization above change the accuracy, since the maximum output of the softmax is unchanged in both cases. However, SE-SNN enables better calibration by allowing the temperature scaling to depend on the per-instance variance, so hard-to-classify high-variance instances are scaled more strongly.

Training SE-SNN We train SE-SNN using a combination of the standard cross-entropy L_{cl} loss and our max-entropy regularizer Ω as

$$L = L_{cl} + \omega \Omega(\theta), \quad (9)$$

where ω balances the classification loss and max-entropy regularizer and Ω is defined according to the different applications as discussed above.

Experiments

Experiments are carried out to evaluate the efficacy of the proposed framework in four applications: neural network pruning, adversarial attack defense, learning with label noise, and model calibration.

Neural Network Pruning

Competitors We follow the architecture/dataset combinations used in most recent neural network pruning studies, including LeNet-5-Caffe¹ network on MNIST (LeCun et al. 1998), VGG-16 (Simonyan and Zisserman 2015) on CIFAR10 (Krizhevsky and Hinton 2009) and a variant of VGG-16 on CIFAR100. Under these settings, the proposed method is compared with the following contemporary state-of-the-art methods including Generalized Dropout (GD) (Srinivas and Babu 2016), Group Lasso (GL) (Wen et al. 2016), Sparse Variational Dropout (VD) (Molchanov, Ashukha, and Vetrov 2017), Structured Bayesian Pruning (SBP) (Neklyudov et al. 2017), Bayesian Compression with Group Normal Jeffreys Prior (BC-GNJ) and Group Horseshoe Prior (BC-GHS) (Louizos, Ullrich, and Welling 2017), Sparse L0 Regularization (L0) and L0 with separate λ for each layer (L0-sep) (Louizos, Welling, and Kingma 2018), Variational Information Bottleneck (VIBNet) (Dai et al. 2018), and Network Slimming (NS) (Liu et al. 2017).

Evaluation Metrics Following the majority of the existing evaluations, we monitor the test error while focusing on the following three compression/complexity metrics: (a) Model size: The ratio of nonzero weights in the compressed network versus the original model. (b) FLOPs: The number of floating point operations required to predict a label from an input image during test². (c) Run-time memory footprint: The ratio of the space for storing hidden feature maps during run-time in the pruned network versus the original network.

Training While many existing studies remove redundant *weights*, we remove redundant *neurons* during compression. Therefore, we can use the pipeline proposed in (Molchanov et al. 2017). Specifically, after training the neural network, an initial batch pruning stage is followed by a loop of removing the least important neuron and fine-tuning. Since SE-SNN is designed to discount unimportant neurons through inflating their *pre-activation* variance, we find that the network achieves this by simultaneously assigning high-variance and negative mean. As a result, a large portion of redundant neurons never activate their RELU non-linearity. In the initial batch pruning stage, neuron inactivity thus provides a single-step pruning criterion before the iterative pruning begins (and one that is guaranteed not to affect the test accuracy since these neurons propagate no information). The pruning then enters the second stage where the least important neuron removal + fine-tuning loop continues until reaching the target trade-off between accuracy and model compression

¹<https://github.com/BVLC/caffe/tree/master/examples/mnist>

²Following the setting in (Dai et al. 2018), we count each multiplication as a single FLOP and exclude additions, which is also consistent with most prior work

Methods	Model size (%)	FLOPs (Mil)	Memory (%)	Error (%)
GD (Srinivas and Babu 2016)	1.38	0.250	32.00	1.1
GL (Wen et al. 2016)	23.69	0.201	19.35	1.0
VD (Molchanov, Ashukha, and Vetrov 2017)	9.29	0.660	60.78	1.0
SBP (Neklyudov et al. 2017)	19.66	0.213	21.15	0.9
BC-GNJ (Louizos, Ullrich, and Welling 2017)	0.95	0.283	35.03	1.0
BC-GHS (Louizos, Ullrich, and Welling 2017)	0.64	0.153	22.80	1.0
L0 (Louizos, Welling, and Kingma 2018)	8.92	1.113	85.82	0.9
L0-sep (Louizos, Welling, and Kingma 2018)	1.08	0.389	40.36	1.0
VIBNet (Dai et al. 2018)	0.83	0.094	15.55	1.0
SE-SNN	2.35	0.061	11.08	0.9

Table 1: Compression results on MNIST using LeNet-5-Caffe

Methods	Model size (%)	FLOPs (Mil)	Memory (%)	Error (%)
BC-GNJ (Louizos, Ullrich, and Welling 2017)	6.57	141.50	81.68	8.6
BC-GHS (Louizos, Ullrich, and Welling 2017)	5.40	121.90	74.82	9.0
VIBNet (Dai et al. 2018)	5.30	70.63	49.57	8.8(8.5)
SE-SNN	2.57	53.61	49.41	8.0

Table 2: Compression results on CIFAR10 using VGG16

objectives. Since one neuron/channel is removed at each iteration, the accuracy never drops sharply. We set the regularizer weight ω (Eq. 9) and margin b (Eq. 5) as 0.01 and 4 in all experiments, respectively.

Testing During testing, only the mean of the learned distribution is passed between layers. So there are no additional parameters and inference cost compared to the network’s deterministic counterpart. The distribution generation branches are only used during training to identify redundant neurons.

Results on MNIST The most commonly used benchmark and architecture is MNIST with LeNet-5-Caffe. We follow the standard training and testing protocols. The results are shown in Table 1. It is clear that SE-SNN achieves the best performance on FLOPs, run-time memory footprint, and test error. In terms of model size, our model is only comparable with the state of art. This is because it does not prune the linear layer as much as other methods. Instead, it focuses on pruning the convolutional filters, hence the excellent performance on FLOPS and memory footprint.

Results on CIFAR10 For CIFAR10, several VGG16 variants and training protocols have been proposed in different works. We use the standard VGG16 architecture but change the dimension of linear layers from 4096 to 512, as in (Louizos, Ullrich, and Welling 2017; Dai et al. 2018). The results in Table 2 compare our method with (Louizos, Ullrich, and Welling 2017; Dai et al. 2018). The error rate for VIBNet in parentheses was obtained by further fine-tuning the pruned architecture. Our model achieves the best performance across all the evaluation metrics and error rates.

Results on CIFAR100 We compare with the study in (Liu et al. 2017), which uses the same VGG16 variant that replaces two fully connected layers with three convolutional layers.

This architecture improves accuracy at the expense of FLOPs and memory. The results in Table 3 show that our model produces the best compression result while maintaining comparable accuracy. Note that the 26.2% error rate achieved by our model is identical to that of the original network. So if the accuracy drop is used as a compression metric, our model is as good as any competitor.

Adversarial Defense

Experimental Setting Following (Alemi et al. 2017), we focus on two types of adversarial attacks: Fast Gradient Sign (FGS) (Goodfellow, Shlens, and Szegedy 2015) and an optimization-based attack CW-L2 (Carlini and Wagner 2017; Athalye, Carlini, and Wagner 2018). We evaluate untargeted FGS attacks with attack magnitude ϵ ranging from 0.0 to 0.5 and untargeted CW-L2 attack on models trained on MNIST. We use a popular architecture including three FC layers with 1024, 1024 and 256 output neurons respectively. The third FC layer is implemented as a stochastic layer. We set the regularizer weight and margin as 0.1 and 16, respectively. Results averaged over 20 runs are reported. We compare against the original (undefended) network, termed as ‘Baseline’, Deep VIB (Alemi et al. 2017) using the variational information bottleneck, Bayesian Adversarial Learning (BAL) (Ye and Zhu 2018) putting a distribution on the adversarial data-generating process, Adv-BNN (Liu et al. 2019) learning a BNN to incorporate the effective randomness and using adversarial training to seek the best model distribution, and RSE (Liu et al. 2018) adding random noise layer in the neural network and using the ensemble prediction over random noises to stabilise the performance. Since our model is stochastic, we follow the best practice recommended in (Athalye, Carlini, and Wagner 2018) for attacking stochastic

Methods	Model size (%)	FLOPs (Mil)	Memory (%)	Error (%)
NS-Single (Liu et al. 2017)	24.90	250.50	-	26.5
NS-Best (Liu et al. 2017)	20.80	214.80	-	26.0
VIBNet (Dai et al. 2018)	15.08	203.10	73.80	25.9(25.7)
SE-SNN	14.93	181.31	70.16	26.2

Table 3: Compression results on CIFAR100 using a VGG16 variant

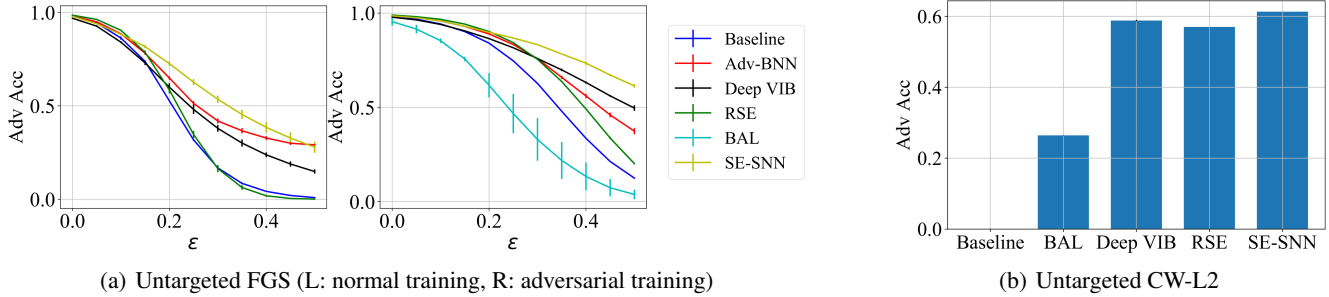


Figure 2: Adversarial defense accuracy (mean + standard deviation) under untargeted FGS and CW attacks on MNIST

models: We compute the expected gradient over multiple stochastic samples for each input when constructing attacks. This is because using the expected gradient over multiple posterior samples produces a better gradient estimator for the attacker, allowing it to generate samples that are much harder to defend against (Athalye, Carlini, and Wagner 2018). For the FGS attack, we evaluate two settings, namely normal training and adversarial training, the latter of which generates and uses FGS attack samples during training to increase adversarial robustness.

Results From the comparative results shown in Figure 2(a), we can see that our SE-SNN outperforms the competing defense methods over a range of FGS attack strengths when trained with adversarial attack samples (Figure 2(a)(R)) or with only normal samples (Figure 2(a)(L)). The advantage of SE-SNN is particularly pronounced when the attack magnitude is large. We can also see from Figure 2(b) that under the stronger CW attacks the Baseline now fails completely. In this case SE-SNN provides the most effective defense. It is worth pointing out that, unlike BAL (Ye and Zhu 2018), which learns their models with explicit adversarial sampling, our SE-SNN can also work with training with only normal samples - Figure 2(a) suggests that our SE-SNN beats BAL even without being trained with adversarial attack samples (comparing SE-SNN in Figure 2(a)(L) to BAL in Figure 2(a)(R)). It is noteworthy that though RSE provides strong defense against CW attacks, it performs very poorly on FGS. We attribute this to the fact that their fixed variance parameter is hand picked for CW. We tried re-tuning their σ for FGS, but failed. In contrast, our SE-SNN’s ability to learn the variance σ leads to good performance against both attacks, and our regularizer hyperparameters ω, β are fixed across all adversarial experiments, without the need for re-tuning for different attacks.

Robustness against Label Noise

Experimental Setting The label noise robustness of our SE-SNN is evaluated on digit recognition using MNIST. We consider patterned label noise which is more common in practice. Specifically, one class label will be flipped to a different one at a strength p . The flipping pattern is fixed for each run but varies across different runs following (Hendrycks et al. 2018). For the network architecture, we follow (Patrini et al. 2017) to train a neural network with two FC layers of dimension 128, where both FC layers are implemented as stochastic layers. Due to the randomness of noisy samples in selection and label reassignment, multiple runs of experiments are conducted.

Competitors *Baseline*: This is the original network without adding our stochastic layers. *Bootstrap_hard* and *Bootstrap_soft* (Reed et al. 2015): Both iteratively use the model-predicted labels to refine the original labels that are potentially corrupted by noise. They differ in whether the updated label is binary or continuous. *Forward Correction* (Patrini et al. 2017): This model predicts the label corruption matrix (capturing the label flipping pattern) by first training a classifier on the noisy labels followed by corruption matrix estimation using the resulting softmax probabilities. The estimated matrix is then employed to regularize the retraining of the model. We stick to the original setting, which uses the *argmax* at the 97th percentile of softmax probabilities for label noise detection. Note that unlike the competitors, our SE-SNN does not have any additional steps to refine the label or prune the training samples. Having said that, it can be easily combined with a label refinement procedure such as the corruption matrix based one in Forward Correction (Patrini et al. 2017).

Results The results are shown in Figure 3. We can see that even without any label correction/refinement procedure, our